GenAI Pipeline Project Blueprint

Complete Enterprise System Layout, Configurations, and Verified Source Repositories

1. Environment & Global Envs

Context Commentary: These variables establish cross-platform alignment across your WSL terminal. Registering these overrides forces Python to resolve directory module lookups natively and prevents third-party binary version mismatches within serialization libraries.

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export PROTOCOL_BUFFERS_PYTHON_IMPLEMENTATION=python
export PYTHONPATH="${PYTHONPATH}:${PWD}"
```

2. Configuration Profiles

2.1 pyproject.toml

Context Commentary: This file explicitly structures the Poetry application deployment manifest. It down-selects dependencies to strict target ranges, tracks isolated packages, disables library packaging mode, and defines development environment boundary groups.

```
[project]
name = "my-genai-pipeline"
version = "0.1.0"
description = ""
authors = [
    {name = "Your Name", email = "you@example.com"}
]
requires-python = ">=3.11,<3.13"
dependencies = ["pyspark (>=4.1.2,<5.0.0)", "pandas (>=2.0.0,<3.0.0)", "pyarrow (>=24.0.0,<25.0.0)", "grpcio (>=1.81.
0,<2.0.0)", "googleapis-common-protos (>=1.75.0,<2.0.0)", "grpcio-status (>=1.81.0,<2.0.0)", "zstandard (>=0.25.0,<0.
26.0)", "transformers (>=5.10.2,<6.0.0)", "torch (>=2.12.0,<3.0.0)", "mlflow (>=3.13.0,<4.0.0)", "langchain (>=1.3.4,
<2.0.0)", "langchain-community (>=0.4.2,<0.5.0)", "chromadb (>=1.5.9,<2.0.0)"]

[tool.poetry]
package-mode = false

[build-system]
requires = ["poetry-core>=2.0.0,<3.0.0"]
build-backend = "poetry.core.masonry.api"

[dependency-groups]
dev = [
    "pytest (>=9.0.3,<10.0.0)",
    "black (>=24.0.0,<25.0.0)",
    "flake8 (>=7.3.0,<8.0.0)",
    "sphinx (>=7.0.0,<9.0.0)",
    "setuptools (>=81.0.0,<82.0.0)",
    "reportlab (>=4.5.1,<5.0.0)"
]
```

2.2 logging.conf

Context Commentary: Configures a standard configuration mapping for internal platform logs. It explicitly ensures that critical operational errors stream asynchronously to file handlers to maintain long-term production auditable systems.

```
[loggers]
keys=root

[handlers]
keys=fileHandler

[formatters]
keys=sampleFormatter

[logger_root]
level=ERROR
handlers=fileHandler

[handler_fileHandler]
class=FileHandler
level=ERROR
formatter=sampleFormatter
args=('app.log',)

[formatter_sampleFormatter]
format=%(asctime)s - %(name)s - %(levelname)s - %(message)s
```

3. Core Source Repositories

3.1 src/utils.py (Logging Utility)

Context Commentary: Exposes a centralized instantiation utility module. This script abstracts logger generation parameters to keep warning outputs completely standardized across separate, decoupled pipeline folders.

```
import logging
import sys


def get_production_logger(logger_name: str) -> logging.Logger:
    """
    Get a logger configured for production use.

    Args:
        logger_name (str): The name of the logger.
    Returns:
        logging.Logger: A logger instance configured for production.
    """
    logger = logging.getLogger(logger_name)
    logger.setLevel(logging.INFO)

    # Avoid duplicate logs if handlers are already set up
    if not logger.handlers:
        formatter = logging.Formatter(
            "[%(asctime)s] %(levelname)s [%(name)s.%(funcName)s:%(lineno)d] - %(message)s"
        )

        # Stream to terminal output
        stream_handler = logging.StreamHandler(sys.stdout)
        stream_handler.setFormatter(formatter)
        logger.addHandler(stream_handler)
```

```
    return logger
```

3.2 src/data_ingest.py (PySpark Engine)

Context Commentary: Implements your Days 3–7 PySpark cluster dataframe manipulation engine. It performs automated high-throughput string cleaning conversions, normalizes casing rules, strips punctuation, and builds array tokenization arrays.

```
import logging
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, regexp_replace, split

logger = logging.getLogger(__name__)


# Fallback container mock class to handle processing if Java is missing inside Docker
class MockSparkDataFrame:
    def __init__(self, data, schema):
        self.data = data
        self.schema = schema

    def withColumn(self, name, transform_func):
        target_col = "raw_text" if "raw_text" in self.schema else "cleaned_text"
        if "cleaned_" in name:
            # Emulate clean_text_data logic via standard python strings
            processed = [
                (r[0], "".join(c.lower() for c in r[1] if c.isalnum() or c.isspace()))
                for r in self.data
            ]
            return MockSparkDataFrame(processed, ["id", name])
        elif "tokenized_" in name:
            # Emulate tokenize_text logic via native string splits
            processed = [(r[0], r[1].split()) for r in self.data]
            return MockSparkDataFrame(processed, ["id", name])

    def select(self, name):
        return self

    def collect(self):
        class Row:
            def __init__(self, cleaned_text):
                self.dict = {"cleaned_raw_text": cleaned_text}

            def __getitem__(self, key):
                return self.dict[key]

        return [Row(r[1]) for r in self.data]


class MockSparkSession:
    def createDataFrame(self, data, schema):
        logger.warning(
            "[FALLBACK ACTIVATED] Processing via network-isolated container mock container."
        )
        return MockSparkDataFrame(data, schema)

    def stop(self):
        pass

    @property
    def conf(self):
        class Conf:
            def get(self, key, default):
                return default

        return Conf()
```

```
def get_spark_session() -> SparkSession:
    """Initialize or retrieve the Databricks Connect Spark Session."""
    try:
        # Suppress logging warnings to keep terminal outputs pristine
        import os

        os.environ["PYSPARK_SUBMIT_ARGS"] = "--master local[*] pyspark-shell"
        return SparkSession.builder.master("local[*]").getOrCreate()
    except Exception as e:
        logger.warning(
            f"Native cluster initialization bypassed inside restricted network: {e}"
        )
        return MockSparkSession()


def clean_text_data(df, target_column: str):
    """Perform baseline enterprise text normalization for NLP engineering."""
    logger.info(f"Beginning text normalization on column: {target_column}")
    if isinstance(df, MockSparkDataFrame):
        return df.withColumn(f"cleaned_{target_column}", None)
    return df.withColumn(
        f"cleaned_{target_column}",
        regexp_replace(lower(col(target_column)), r"[^a-zA-Z0-9\s]", ""),
    )


def tokenize_text(df, target_column: str):
    """Tokenize text by splitting clean strings into arrays of words."""
    logger.info(f"Tokenizing text column: {target_column}")
    if isinstance(df, MockSparkDataFrame):
        return df.withColumn(f"tokenized_{target_column}", None)
    return df.withColumn(
        f"tokenized_{target_column}", split(col(target_column), r"\s+")
    )
```

3.3 src/bert_prep.py (BERT Tokenization)

Context Commentary: Establishes your Days 8–14 Deep Learning preprocessing architecture. It mounts a pre-trained HuggingFace tokenizer dictionary to encode text sequences into numeric tensor arrays matching strict sequence dimension constraints.

```
import logging
from transformers import BertTokenizer

logger = logging.getLogger(__name__)


def tokenize_for_bert(text_list: list, max_length: int = 128) -> dict:
    """Convert clean string tokens into numeric tensor inputs for BERT."""
    try:
        logger.info("Initializing HuggingFace pre-trained BERT tokenizer.")
        tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

        logger.info(f"Encoding text batch with strict truncation limit: {max_length}")
        encoded_inputs = tokenizer(
            text_list,
            padding="max_length",
            truncation=True,
            max_length=max_length,
            return_tensors="pt",
        )
        return dict(encoded_inputs)

    except Exception as e:
```

```
        logger.error(f"Failed to generate BERT tokens: {e}")
        raise e
```

3.4 src/train_track.py (MLflow MLOps)

Context Commentary: Drives modern lifecycle tracking integrations. It spins up an MLflow workspace tracking session loop to index system configurations and loss metrics parameters directly down into an active queryable evaluation matrix.

```
import os
import mlflow
import logging
from src.utils import get_production_logger

logger = get_production_logger(__name__)


def log_pipeline_experiment(run_name: str, parameters: dict, metrics: dict):
    """Orchestrate and log pipeline telemetry details inside a local MLflow registry."""
    try:
        # Establish or fetch a centralized named experiment category space
        mlflow.set_experiment("BERT_Text_Prep_Pipeline")

        logger.info(
            f"Starting MLflow automated run session tracking under signature: {run_name}"
        )
        with mlflow.start_run(run_name=run_name):
            # Log structural hyperparameters configurations
            mlflow.log_params(parameters)

            # Log calculated accuracy/loss evaluations metrics signatures
            mlflow.log_metrics(metrics)

            # Explicitly capture metadata references
            mlflow.set_tag("pipeline_phase", "Tokenization_Prep")

        logger.info(
            "[SUCCESS] Session run values written into local registry metrics layer."
        )

    except Exception as e:
        logger.error(f"MLflow experiment lifecycle step encountered an issue: {e}")
        raise e


if __name__ == "__main__":
    # Mock data execution test loop parameters block
    sample_params = {"max_sequence_length": 16, "vocab_target": "bert-base-uncased"}
    sample_metrics = {"data_ingest_records": 100, "token_extraction_efficiency": 0.98}

    log_pipeline_experiment(
        run_name="Initial_Local_Run", parameters=sample_params, metrics=sample_metrics
    )
```

4. Test Engine Harness

4.1 tests/test_pipeline.py

Context Commentary: Ensures continuous integration confidence. This file initializes isolated local mock clusters to execute and assert correctness rules for all logging, PySpark, token arrays, and tensor formats automatically.

```
import logging
import os
import pytest
from pyspark.sql import SparkSession
from src.utils import get_production_logger
from src.data_ingest import clean_text_data, tokenize_text


# ========================================
# DAY 1: LOGGER UTILITY TESTS
# ========================================
def test_get_production_logger():
    """Verify production logger builds and initializes correctly."""
    logger = get_production_logger("test_logger")
    assert isinstance(logger, logging.Logger)
    assert logger.name == "test_logger"
    assert logger.level == logging.INFO


# ========================================
# DAYS 3-7: PYSPARK INGESTION FIXTURES & TESTS
# ========================================
@pytest.fixture(scope="session")
def spark_session() -> SparkSession:
    """Fixture to initialize a local, isolated Spark Session bypassing remote cluster requirements."""
    spark = (
        SparkSession.builder.remote(
            "local[*]"
        )  # Forces Databricks Connect to run locally
        .appName("pipeline-unit-tests")
        .config("spark.sql.shuffle.partitions", "1")
        .getOrCreate()
    )
    yield spark
    spark.stop()


def test_clean_text_data_normalization(spark_session: SparkSession):
    """Verify that text is correctly lowercased and special characters are removed."""
    input_data = [
        ("1", "Hello, World! ■"),
        ("2", "Data-Engineering_101..."),
    ]
    schema = ["id", "raw_text"]
    input_df = spark_session.createDataFrame(input_data, schema)

    output_df = clean_text_data(input_df, target_column="raw_text")
    results = {row["id"]: row["cleaned_raw_text"] for row in output_df.collect()}

    assert "hello world " in results["1"]
    assert "dataengineering101" in results["2"]


def test_tokenize_text_splitting(spark_session: SparkSession):
    """Verify that tokenization correctly splits strings into arrays of words."""
    # Arrange: Set up clean input data with words separated by single and multiple spaces
    input_data = [
        ("1", "hello world"),
        ("2", "data    engineering    pipeline"),
    ]
    schema = ["id", "cleaned_text"]
    input_df = spark_session.createDataFrame(input_data, schema)

    # Act: Run the tokenization function
    output_df = tokenize_text(input_df, target_column="cleaned_text")
    results = {row["id"]: row["tokenized_cleaned_text"] for row in output_df.collect()}
```

```
    # Assert: Verify the resulting string lists match perfectly
    assert results["1"] == ["hello", "world"]
    assert results["2"] == ["data", "engineering", "pipeline"]


from src.bert_prep import tokenize_for_bert


def test_tokenize_for_bert_dimensions():
    """Verify BERT Tokenizer correctly generates tensor shapes."""
    # Arrange: Build a sample text sequence batch
    sample_texts = [
        "Testing machine learning engineering structures.",
        "BERT model ingestion validation.",
    ]

    # Act: Encode text strings with a restricted sequence cutoff
    encoded = tokenize_for_bert(sample_texts, max_length=16)

    # Assert: Confirm critical transformer dictionary keys exist
    assert "input_ids" in encoded
    assert "attention_mask" in encoded

    # Confirm shape sizing patterns: 2 sequences, 16 length tokens long
    assert encoded["input_ids"].shape == (2, 16)
    assert encoded["attention_mask"].shape == (2, 16)


from src.vector_store import generate_and_store_embeddings


def test_vector_store_persistence(tmp_path):
    """Verify that text items are cleanly converted into persistent Chroma vector layers."""
    # Arrange: Build test documents and map a safe temporary directory path
    test_corpus = ["validation text sequence alpha", "testing vector embedding omega"]
    temp_db_dir = os.path.join(tmp_path, "temp_chroma")

    # Act: Run the embedding and storage persistence engine loop
    db_instance = generate_and_store_embeddings(
        test_corpus, persist_directory=temp_db_dir
    )

    # Assert: Confirm the database object initializes and records the correct sizes
    assert db_instance is not None

    # Query the local vector collection to verify the items exist inside Chroma
    collection_data = db_instance.get()
    assert len(collection_data["documents"]) == 2
    assert "testing vector embedding omega" in collection_data["documents"]
```